Trabalho I

A Entrada

A entrada do algoritmo será basicamente um conjunto de formas geométricas básicas (retângulos, círculos, etc) dispostos numa região do plano cartesiano.

Considere a Ilustração 1. Cada forma geométrica é definida por uma coordenada âncora (marcada, na figura, por um pequeno ponto vermelho) e por suas dimensões. A coordenada âncora do círculo é o seu centro e sua dimensão é definida por seu raio (r, na figura). A coordenada âncora do retângulo é seu canto inferior esquerdo[1] e suas dimensões são sua largura (w) e sua altura (h). A coordenada âncora de um texto, normalmente, é o início do texto, porém, pode ser definida como o meio ou o fim do texto. Por fim, uma linha é determinada por duas âncoras em suas extremidades. As coordenadas que posicionam as formas geométricas são valores reais.

Cada forma geométrica é identificada por um número inteiro.

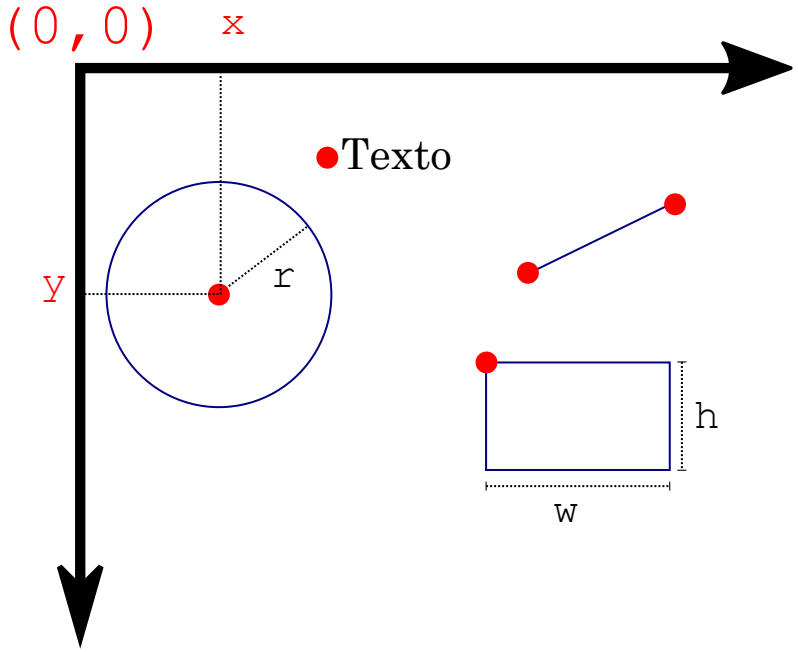


Ilustração 1: Formas no plano

As tabelas abaixo mostram os formatos dos arquivos de entrada (.geo e .qry). Cada comando tem um certo número de parâmetros. Os parâmetros mais comuns são:

- `i,j,k`: número inteiro, maior ou igual a 1. Identificador de uma forma geométrica.
- `p`: número inteiro positivo ([1,10]). Identifica um polígono.
- `r`: número real. Raio do círculo.
- `x,y`: números reais. Coordenada (x,y).
- `cor`: string. Cor válida dentro do padrão SVG.[2]

1 Note que o plano cartesiano está desenhado "de ponta-cabeça" em relação à representação usual.
2 http://www.december.com/html/spec/colorsvg.html.
https://www.w3.org/Graphics/SVG/IG/resources/svgprimer.html

comando	parâmetros	descrição
c	`i x y r corb corp`	*desenhar círculo.* corb *é a cor da borda e* corp *é a cor do preenchimento*
r	`i x y w h corb corp`	*desenhar retângulo: w é a largura do retângulo e h, a altura.* corb *é a cor da borda e* corp *é a cor do preenchimento*
l	`i x1 y1 x2 y2 cor`	*Desenhar linha com extremidades nos pontos (x1,y1) e (x2,y2), com a cor especifiicada.*
t	`i x y corb corp a txto`	*desenha o texto* txto *nas coordenadas (**x,y**) e com a cores indicadas.* corb *é a cor da borda e* corp *é a cor do preenchimento. O parâmetro **a** determina a posição da âncora do texto: **i**, no início; **m**, no meio, **f**, no fim.* *O texto txto é o último parâmetro do comando. Pode incluir espaços em branco e se estende até o final da linha.*
ts	`fFamily fWeight fSize`	*Muda o estilo dos textos (comando t) subsequentes.* *font family: sans (sans-serif), serif, cursive;* *font weight (n: normal, b: bold, b+: bolder, l: | lighter)*
comandos.geo		

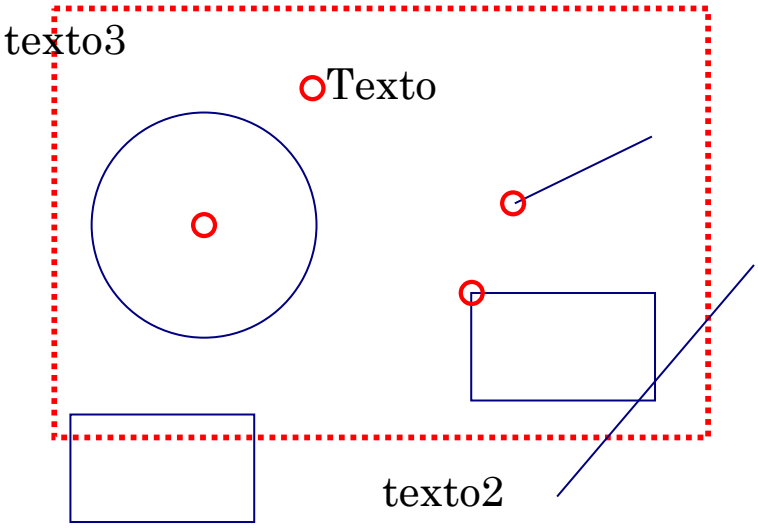
Algumas consultas fazem referência a filas e listas. O programa deve manter, pelo menos 1 fila para o polígono corrente e uma lista para as figuras selecionadadas.

comando	parâmetros	descrição
inp	`p i`	*Insere no polígono* p *as coordenadas da âncora da figura identificada por* i. *No caso de linhas, insere a extremidade de menor x (se empate, de menor y)* ***TXT****. Reportar a coordenada inserida e os dados da respectiva figura*
rmp	`p`	*Remove a coordenada mais antiga inserida no poligono p.* ***TXT****. Reportar a coordenada removida*
pol	`p i d corb corp`	*Produz um conjunto de linhas (e os insere no "banco de dados") relativas ao polígono* p. *As linhas produzidas correspondem à borda do polígono e as linhas de seu preenchimento. Os identificadores das linhas produzidas são sequenciais, a partir de* i. *O parâmetro* d *é a distância entre as linhas do preenchimento;* corb, *a cor das linhas do polígono;* corp, *a cor das linhas do preenchimento.*
clp	`p`	*Remove todas as coordenadas do polígono p.*
sel	`x y w h`	*Seleciona as figuras inteiramente dentro da região especificada pelos parâmetros. (Desconsidera seleções anteriores)* *SVG: desenhar região (borda vermelha pontilhada). Desenhar um anel vermelho em volta da âncora das figuras selecionadas.* *TXT: reportar identificador e tipo das figuras selecionada*
dels		*Remove do "banco de dados" todas as figuras selecionadas.* *SVG: As figuras removidas naturalmente não aparecerão no SVG (pois, não existem mais) Marcar com um pequeno x vermelho a âncora da figura removida* *TXT: Reportar o* id *e os dados das figuras removidas*
mcs	`dx dy corb corp`	*Modifica a posição e as cores das formas selecionadas: são transladadas de* dx *e* dy*; cor da borda é alterada para* corb *e cor do preenchimento (se aplicável) é alterada para* corp .
Comandos .qry		

A Saída

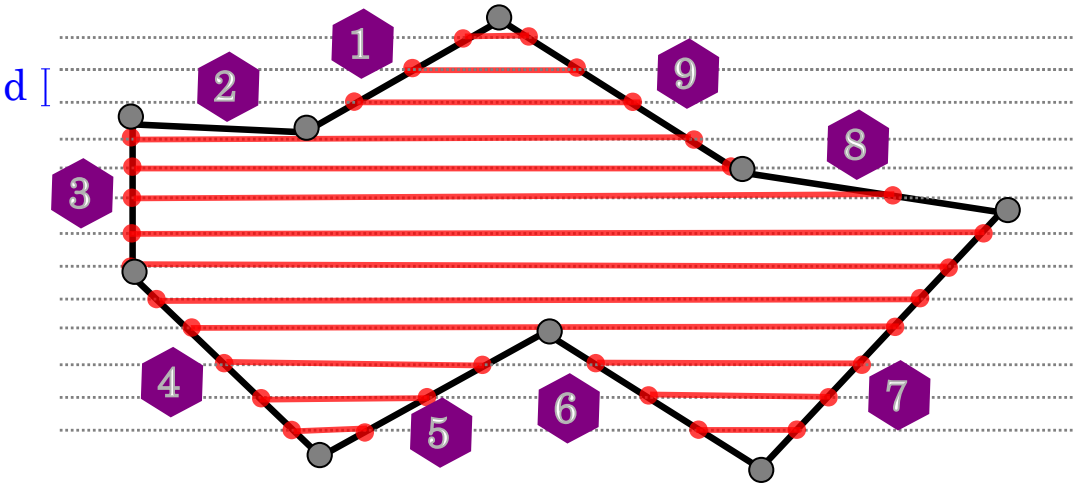
A saída deve estar de acordo com a descrição dos comandos. Vale a pena ilustrar alguns destes comandos.

O comando **`sel`** deve mostrar a região de seleção e a âncora das figuras selecionadas.



A figura abaixo ilustra o efeito do comando **`pol`**. Os pontos cinzas correspondem às coordenadas produzidas pelo comando **`inp`**. Os segmentos pretos são as bordas do polígono determinado pela ordem de execução de comandos **`inp`**. Estes segmentos devem ser acrescentados ao "banco de dados". Os segmentos vermelhos constituem o preenchimento do polígono. Também devem ser inseridos no "banco de dados". Porém, os pontos vermelhos e as linhas tracejadas cinzas são apenas explicativas e não devem ser inseridas no "banco de dados".

O algoritmo para o preenchimento do polígono faz uma linha horizontal (cinza tracejada na figura) "andar" sobre o polígono em saltos de distância **`d`**. As intersecções desta linha com segmentos das bordas do polígono (ponto vermelho) determinam dos extremos das linhas de preenchimento.



9 lados + 16 linhas preenchimento = 25 linhas inseridas

IMPLEMENTAÇÃO

As estruturas de dados lista e fila devem ser implementadas conforme o TAD mostrado em aula.

Os polígonos devem ser armazenados em filas. As formas devem ser armazenadas em listas.

É **terminantemente proibido** definir structs nos arquivos de cabeçalho (.h).

O programa deve estar bem modularizado (arquivos .h e .c). Cada estrutura de dados deve estar em um módulo separado. O arquivo .h deve estar muito bem documentado (lembre-se que é um "contrato").

Obs.: usar implementação dinâmica e encadeamento duplo para lista; implementação estática (circular) para as filas.

Para cada módulo, escrever um arquivo de teste unitário usando o módulo unity (conforme descrição geral. O makefile deve conter targets para compilação e execução dos testes unitários.

AVALIAÇÃO

Espera-se uma atitude pró-ativa para a aquisição dos conhecimentos (i.e., estudo) para resolver o problema proposto.

A avaliação consistirá da execução dos testes e da inspeção de código. A nota é dividida em 2 partes: 1.0, entrega do .h e testes unitários do poligono; 9.0, proporcional ao número de testes bem-sucedidos, aplicados os descontos descritos abaixo.

ATENÇÃO: Caso algum tipo de FRAUDE seja detectada: Nota **ZERO** a **TODOS** os envolvidos.

Critério	Desconto
Definir struct em arquivo .h	2.5
Modularização Pobre: .h mal projetado, mal documentado	até 1.5
Não implementado conforme especificado	tipicamente até 4.5, mas em casos graves, o desconto pode ser total
• não implementar estruturas pedidas ou implementação não funcional (gravíssimo)	
Procedimentos extensos e/ou complicados	até 1.0
Makefile não funcional	Caso não produza executável: veja "Erro de Compilação"
Poucos commits ou commits concentrados perto do final do prazo.	tipicamente até 3.0; porém, o padrão de commits pode indicar possível fraude.
Implementação ineficiente e/ou desidiosa.	Até 1.5.
Módulos sem respectivos arquivos de testes unitários, ou testes unitários mal-feitos.	Até 2.5
Makefile sem targets (funcionais) para produzir e executar os casos de teste	Até 1.5

Erro de compilação	Nenhum teste será executado. Portanto, nota Zero, especialmente se for causado por negligência do aluno.

O Que Entregar

Os arquivos-fontes devem estar em um repositório GIT. Submeter ao Classroom um arquivo-texto (.txt) com uma linha, com o formato abaixo. A URL fornecida será usada para clonar o repositório (git clone)

`apelido url-do-repositorio`
Exemplo: `joseMane https://github.com/zemane/t2.git`

RESUMO DOS PARÂMETROS DO PROGRAMA TED

Parâmetro / argumento	Opcional	Descrição
`-e` ***path***	S	Diretório-base de entrada (**BED**)
`-f` ***arq*****.geo**	N	Arquivo com a descrição da cidade. Este arquivo deve estar sob o diretório **BED**.
`-o` ***path***	N	Diretório-base de saída (**BSD**)
`-q` ***arqcons*****.qry**	S	Arquivo com consultas. Este arquivo deve estar sob o diretório **BED**.

RESUMO DOS ARQUIVOS PRODUZIDOS

-f	-q	comando com sufixo	arquivos
arq.geo			arq.svg
arq.geo	***arqcons***.qry		arq.svg arq-arqcons.svg arq-arqcons.txt
arq.geo	***arqcons***.qry	***sufx***	arq.svg arq-arqcons.svg arq-arqcons.txt arq-arqcons-sufx.[svg|txt][3]

ATENÇÃO:
* os fontes devem ser compilados com a opção `-fstack-protector-all`.
* adotamos o padrão C99. Usar a opção `-std=c99`.

3 Podem ser produzidos os respectivos arquivos .svg e/ou .txt, dependendo da especificação do comando.